Git Guild 模块划分与职责边界说明

1. 文档目的

本文件用于说明 Git Guild 在 P2 阶段的模块划分方案、优先级分级以及各模块的职责边界，为后续的架构图绘制、接口设计、数据库设计与 ADR 编写提供依据。

Git Guild 是一个面向开源初学者的游戏化代码托管与悬赏协作平台。系统需要同时服务两类主要用户：

- 初学者：浏览任务、接取任务、提交成果、获取成长反馈
- 维护者：发布任务、审核成果、管理协作流程

此外，平台还包含管理员角色，用于任务治理、异常处理与平台规则执行。

2. 总体划分原则

本项目采用前后端分离的分层单体架构，后端在单体内部按领域模块组织。模块划分遵循以下原则：

- 以业务职责而非技术层次作为一级划分依据
- 将“推荐匹配”视为核心业务模块，而非后续附加功能
- 将“任务协作闭环”作为优先级最高的系统主链路
- 将平台治理能力嵌入现有模块中，避免课程项目中过度拆分
- 对接外部代码托管底座，但不自行实现底层 Git 仓库管理能力

3. 核心业务链路

Git Guild 的核心业务链路如下：

1. 用户注册或登录平台
2. 用户浏览任务列表，按条件筛选任务或接收系统推荐
3. 用户查看任务详情、项目结构说明与完成标准
4. 用户接取任务并完成开发
5. 用户提交成果，并与对应 Issue / PR 关联
6. 维护者或管理员进行审核并给出反馈
7. 平台发送状态通知并更新成长记录

围绕以上主链路，系统划分为 8 个主要模块。

4. 模块总览

4.1 模块列表

1. 用户与认证模块
2. 代码托管底座适配模块
3. 任务与悬赏模块
4. 推荐匹配模块
5. 提交与审核反馈模块
6. 新手引导与项目理解模块
7. 通知模块
8. 成长激励模块

4.2 优先级分级

P0：必须完成的核心模块

- 用户与认证模块
- 代码托管底座适配模块
- 任务与悬赏模块
- 推荐匹配模块
- 提交与审核反馈模块
- 新手引导与项目理解模块
- 通知模块

P1：建议在核心闭环完成后实现的增强模块或增强能力

- 成长激励模块的轻量能力（XP、等级、贡献记录）
- 推荐匹配模块增强
- 审核反馈增强
- 通知增强
- 用户中心增强

P2：可后置实现的扩展能力

- 成长激励模块的展示型能力（徽章、排行榜）
- 导师模式
- 社区互动能力
- 高级推荐策略
- 运营统计看板

说明：这里的 P0 / P1 / P2 表示实现优先级，而非课程阶段名称。

5. 各模块职责边界

5.1 用户与认证模块

核心职责

- 平台账号注册、登录、登出
- 用户基本资料维护
- 角色管理：初学者、维护者、管理员
- 平台内权限控制

典型功能

- 用户注册与登录
- 基本个人资料页
- 角色判定与权限校验
- 管理员身份识别

边界说明

- 该模块负责回答“谁可以做什么”
- 不负责任务推荐逻辑
- 不负责任务审核流程本身
- 不负责代码托管底座中的仓库权限实现

5.2 代码托管底座适配模块

核心职责

- 对接 GitHub 与 Gitea 等外部代码托管能力
- 支持 GitHub 仓库导入与单向同步
- 获取仓库、Issue、PR、分支、commit 等元数据
- 支持创建分支、上传变更生成 commit、发起/合并 PR 等平台内 Git 操作
- 接收外部事件或 Webhook
- 为上层业务模块提供统一的仓库能力接口

典型功能

- 仓库接入与仓库信息同步
- Issue / PR 元数据读取
- PR 状态同步
- 外部事件转内部事件

- GitHub 仓库、Issue 与 PR 状态的手动同步
- 创建分支、上传变更并提交 commit
- 发起 Pull Request 与合并 Pull Request

边界说明

- 该模块是适配层，不是业务中心
- 不自行实现 Git 仓库存储与底层托管能力
- 不负责任务规则、成长规则和推荐计算
- GitHub 只作为外部导入与同步来源，平台内协作以 Gitea 为准。
- 不支持 Git Guild / Gitea 到 GitHub 的反向同步。
- 不提供网页代码编辑器，只支持上传变更生成 commit。

5.3 任务与悬赏模块

核心职责

- 任务发布与编辑
- 任务详情展示
- 任务分类、标签、难度分级
- 任务搜索、筛选与排序
- 任务接取与任务状态流转
- 任务冲突处理
- 任务上下架与基础治理动作

典型功能

- 维护者发布任务
- 平台展示任务列表
- 按技术栈、难度、状态、预计时长、新手友好标签等条件筛选
- 任务接取成功 / 接取冲突处理
- 任务状态在“待接取 / 进行中 / 待审核 / 已完成 / 已关闭”等状态之间流转

边界说明

- “任务分类”和“任务筛选”属于本模块核心职责
- 该模块负责“任务本身如何被管理”
- 不负责推荐算法与推荐排序逻辑
- 不负责完整的审核意见记录与审核 SLA 处理

5.4 推荐匹配模块

核心职责

- 为初学者推荐适合接取的任务
- 为维护者推荐合适的贡献者
- 输出推荐理由
- 管理推荐规则与排序逻辑

典型功能

- 基于技术栈匹配推荐任务
- 基于历史行为推荐任务
- 基于成长阶段、难度层级和预计时长推荐任务
- 推荐理由说明，如“与你近期完成的前端任务相似”

边界说明

- 推荐模块负责“系统主动推荐”
- 不负责用户主动筛选
- 不负责任务分类定义
- 不直接修改任务状态或审核状态

说明：推荐与筛选不是同一能力。筛选由用户主动设定条件，推荐由系统主动给出候选结果。

5.5 提交与审核反馈模块

核心职责

- 关联任务与成果提交记录
- 管理审核状态与审核意见
- 支持维护者审核与管理员审核
- 处理审核超时提醒与反馈流程

典型功能

- 将任务与 PR 或提交记录关联
- 维护者审核通过、退回修改、驳回
- 管理员对异常任务、违规任务进行审核、下架或关闭
- 审核超时提醒与状态升级

边界说明

- 该模块负责“成果如何被判定”

- 不负责任务分类和任务筛选
- 不负责仓库底层 PR 能力本身
- 不负责推荐结果生成

说明：管理员对任务的审核属于该模块的重要职责，而不是边缘功能。

5.6 新手引导与项目理解模块

核心职责

- 提供任务说明模板
- 提供项目结构概览
- 提供贡献流程指引
- 提供示例 PR、运行说明、参考资料等辅助内容

典型功能

- 任务发布模板
- 新手任务页面中的“如何开始”指引
- 项目结构视图或模块入口说明
- 参考示例链接与步骤说明

边界说明

- 该模块负责“降低新手进入成本”
- 不负责任务状态流转
- 不负责通知发送
- 不负责成长值计算

5.7 通知模块

核心职责

- 统一处理站内通知与邮件通知
- 对关键状态变化进行消息触达
- 支持通知汇总与通知策略

典型功能

- 任务被接取通知
- 提交待审核通知
- 审核结果通知

- 超时提醒通知
- 每日汇总通知

边界说明

- 该模块不负责业务决策
- 只负责“谁在什么时间收到什么消息”
- 通常由其他业务模块触发通知事件

5.8 成长激励模块

核心职责

- 管理 XP、等级、徽章、贡献记录、排行榜等成长要素
- 根据任务完成和审核结果更新成长数据

典型功能

- 完成任务后增加 XP
- 等级提升
- 记录贡献轨迹
- 徽章授予
- 排行榜展示

边界说明

- 该模块负责“完成行为如何转化为成长反馈”
- 不负责任务分配
- 不负责仓库同步
- 不负责审核主流程

说明：在优先级上，XP、等级和贡献记录应优先于徽章和排行榜。

6. 管理员能力落点

虽然本项目不单独拆分“管理员模块”，但管理员能力必须在系统中明确落位：

- 用户与认证模块：管理员角色与权限控制
- 任务与悬赏模块：管理员可执行任务分类管理、任务上下架、基础治理
- 提交与审核反馈模块：管理员可处理异常审核、违规任务、争议任务
- 通知模块：管理员审核结果与治理动作的通知下发

这种设计能够避免课程项目中过度拆分后台系统，同时保证管理员职责在架构层面可追踪。

7. 模块间关系

7.1 主依赖关系

- 用户与认证模块：被其他模块依赖，用于身份与权限判断
- 代码托管底座适配模块：被任务模块、审核模块、新手引导模块依赖
- 任务与悬赏模块：作为系统主业务中枢，被推荐模块、审核模块、通知模块、成长模块依赖
- 推荐匹配模块：依赖用户数据、任务数据、成长数据
- 提交与审核反馈模块：依赖任务模块与代码托管适配模块
- 新手引导与项目理解模块：依赖任务数据与仓库结构信息
- 通知模块：由任务模块、审核模块、成长模块触发
- 成长激励模块：依赖任务完成与审核通过事件

7.2 主链路与辅助链路

主链路：

- 用户 -> 任务 -> 提交 -> 审核 -> 通知

辅助链路：

- 推荐匹配
- 新手引导
- 成长激励

8. 优先级结论

结合问卷、访谈和课程项目约束，Git Guild 的优先级结论如下：

- P0 应优先保证“找任务、理解任务、接任务、提交成果、收到审核反馈”的完整闭环
- 推荐系统属于 P0 核心能力，而不是后续附加功能
- 管理员任务审核、任务分类和任务筛选都属于核心业务所需能力，不能省略
- P1 重点增强成长反馈、推荐质量、审核效率与通知体验
- P2 再考虑排行榜、徽章、导师模式、社区互动和高级推荐

9. 小结

Git Guild 在 P2 阶段的模块划分应围绕“任务协作闭环 + 推荐匹配核心能力”展开。系统不应将推荐系统、管理员任务审核、任务分类或任务筛选视为边缘能力，而应将其纳入核心业务模块设计中。

该模块划分方案既满足当前课程项目的实现可行性，也为后续的功能增强和架构演进保留了清晰边界。